

《信息安全引论》第一次课程实验报告

刘沛雨 2100012289 2025.11.30

1 密钥加解密实验

1.1 使用不同加密算法进行加密

1.2 学习 ECB 与 CBC 的区别

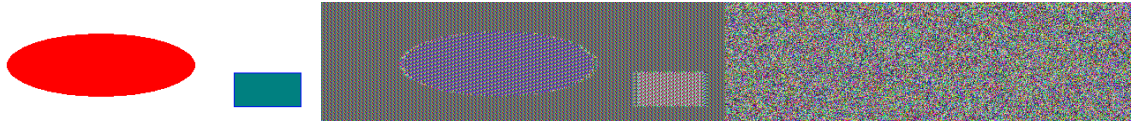


图 1：原图（左）、ECB 加密图（中）、CBC 加密图（右）

ECB 将相同的明文块加密为相同的密文块，因此加密后的图片仍可辨认出原图的形状轮廓。而 CBC 会将当前明文块与先前的密文块异或，使得相同的明文块加密得到的密文不同，因此加密后的图片无法看出原图的形状轮廓。

1.3 损坏的密文

ECB 密文：

`aDvAkNp4nrHpCjxzl kudOGg7wJDaeJ6x6Qo8c5ZLnThoO8CQ2niesekKPHOWS504aDvAkNp4nrHpCjxzl kudOCSrLDSE2Z8jriHGx7T82pw=`

CBC 密文：

`MrcwD2zbOSdst d7TfIWWvvq7yjVn rz5Jl jylPqyZRQPgi6GghPmKiusquqgDT8yFUpCwTwAam+pGIG3O4IFBH6r3GMs0b+0MivTtshW3U4=`

修改了密文的第 30 个字节（大小写相互转换），解码修改后的密文得到：

ECB 明文：

`i love pkuhahah`

`-! "9iS;I7gi love pkuhahah`

`i love pkuhahah`

`i love pkuhahah`

CBC 明文：

`i love pkuhahah`

`␣<jZ,6k,i lovfpkuhahah`

`i love pkuhahah`

`i love pkuhahah`

错误发生在第二个块中。对于 ECB，每个分组独立解密，互不依赖，所以密文块损坏只会导致当前块的明文出现乱码，错误不会扩散到其他块。而对于 CBC，由于解

密需要前一个密文块参与，密文块损坏不仅会导致当前块解密乱码，还会导致下一块明文块的特定位置出现错误。

1.4 密文的填充

输入: *Your Input*

ECB 密文: *BhE2g1QWuKR4BgmKMyPwXg==*

CBC 密文: *olu9vswxN5g3chzzvf8niw==*

明文长度为 10 字节，而 ECB 和 CBC 输出的密文经 Base64 解码后长度均为 16 字节 ($24 \text{ chars} * 3 \text{ bytes} / 4 \text{ chars} - \#'''' = 16 \text{ bytes}$)，这说明加密算法对不足 16 字节的明文进行了填充，将其补齐到了 16 字节。

2 密钥加解密实验

2.1 生成消息摘要和 MAC

MD5(hello.txt) = "8cb64823a52f55342aa3fe6a847219e4"

SHA1(hello.txt) = "cff49558490752a37d2bfd812f593462089ea927"

SHA256(hello.txt) = "3a6e619878f0b201d5e3c92acd581c2ec9c1b4cd63ad4dac984c568f87274c0c"

2.2 Keyed Hash 和 HMAC

HMAC(MD5, kz8F2m9Lp4Q1v7Xj, hello.txt) = "6be72ee0f9e155b56bbbfeac4d74648c"

HMAC(SHA1, R5t8Yu1i9O0p4A2s6D8f, hello.txt) = "cc1a687c129f21673dd3cd95a6b9e4f2597acdfb"

HMAC(SHA256, H7j3K9l2Z1x5C4v8B6n0M3q1W9e4R7tY, hello.txt) =

"53d9bb10f6f6cdf6712ce804a2e35c59030327434da06e9d34ef77ff090ef3c0"

2.3 单向哈希函数的随机性

```
2.3-MD5.txt
1  A[1] = MD5(3)                = eccbc87e4b5ce2fe28308fd9f2a7baf3
2  A[2] = MD5(MD5(3))           = 4bcde37812c15d1d0ae54fc9b3770e8b
3  A[3] = MD5(MD5(MD5(3)))      = 321823f68340096780c21fdceda67914
4
5  Counts = 4480000
6  Bucket 0: *****, Hits=559770
7  Bucket 1: *****, Hits=558844
8  Bucket 2: *****, Hits=558663
9  Bucket 3: *****, Hits=561314
10 Bucket 4: *****, Hits=561612
11 Bucket 5: *****, Hits=561240
12 Bucket 6: *****, Hits=559371
13 Bucket 7: *****, Hits=559186
```

```

≡ 2.3-SHA1.txt
1  A[1] = SHA1(3) = 77de68daecd823babbb58edb1c8e14d7106e83bb
2  A[2] = SHA1(SHA1(3)) = c4e74dddc9cc9e2fcdcb7f63b127fb638831262e
3  A[3] = SHA1(SHA1(SHA1(3))) = 2f1cbe8d5f7f34529bbb60dd42868591567c253e
4
5  Counts = 4580000
6  Bucket 0: *****, Hits=571749
7  Bucket 1: *****, Hits=573418
8  Bucket 2: *****, Hits=572300
9  Bucket 3: *****, Hits=572127
10 Bucket 4: *****, Hits=573367
11 Bucket 5: *****, Hits=571041
12 Bucket 6: *****, Hits=572322
13 Bucket 7: *****, Hits=573676

```

```

≡ 2.3-SHA256.txt
1  A[1] = SHA256(3) = 4e07408562bedb8b60ce05c1decfe3ad16b72230967de01f640b7e4729b49fce
2  A[2] = SHA256(SHA256(3)) = 80903da4e6bbdf96e8ff6fc3966b0cfd355c7e860bdd1caa8e4722d9230e40ac
3  A[3] = SHA256(SHA256(SHA256(3))) = f04ed1ab2cc92969210f295e558fb06a729c79e956bc39a2da5f8cf2ac7bc30f
4
5  Counts = 4900000
6  Bucket 0: *****, Hits=613176
7  Bucket 1: *****, Hits=613903
8  Bucket 2: *****, Hits=611057
9  Bucket 3: *****, Hits=611596
10 Bucket 4: *****, Hits=613416
11 Bucket 5: *****, Hits=612630
12 Bucket 6: *****, Hits=611919
13 Bucket 7: *****, Hits=612303

```

三种算法输出值的低 3 位基本均匀分布在 0 到 7 这 8 个数之间。实验证明了哈希函数的伪随机特性。只要输入（上一轮的哈希值）有微小变化，新产生的哈希值看起来就会是完全随机且不可预测的。

2.4 单向特性

```

≡ 2.4-MD5.txt
1  找到 1 个前导零, MD5(eccbc87e4b5ce2fe28308fd9f2a7baf3) = 4bcde37812c15d1d0ae54fc9b3770e8b
2  花费时间: 0.00 秒
3  找到 2 个前导零, MD5(4bcde37812c15d1d0ae54fc9b3770e8b) = 321823f68340096780c21fdceda67914
4  花费时间: 0.00 秒
5  找到 3 个前导零, MD5(319b819d9ad33970022b0b1a9fac9c3d) = 15ee8fbacb3a386e82c39180cc2301c4
6  花费时间: 0.00 秒
7  找到 6 个前导零, MD5(7c5120fca8e20cd81005911266a61275) = 025cc74503b7912c1c42980ac56a5673
8  花费时间: 0.00 秒
9  找到 9 个前导零, MD5(0298db8732341b60205cb8453b6694b2) = 004099f04b30621b4c4f3ebc3d2a8404
10 花费时间: 0.00 秒
11 找到 11 个前导零, MD5(cd512e7568187d64ae9c4be431827436) = 001b5836e0e937d4abf72ff99d35970e
12 花费时间: 0.01 秒
13 找到 12 个前导零, MD5(c828ab4018dfa5446ac87853e9a1c22a) = 000d510b79be73146ef10eb7bce21fca
14 花费时间: 0.05 秒
15 找到 13 个前导零, MD5(3a8be1cc0b5e53d4683d7c04c324520a) = 00061f68e324aaae570b4370e8d20803
16 花费时间: 0.07 秒
17 找到 15 个前导零, MD5(c475b51773968c11a383ed3fa930cc8a) = 0001e6d558565014f3e383e58b8d02b5
18 花费时间: 0.11 秒
19 找到 16 个前导零, MD5(b0613d90c47f9a81f7be81570771ff4a) = 0000beb6602f10bbf9fe16365ce755e8
20 花费时间: 0.39 秒
21 找到 17 个前导零, MD5(c9288bf9df7504dced485f5f75747deb) = 00004bc2b66025fe33f227ed6244bd25
22 花费时间: 1.97 秒
23 找到 18 个前导零, MD5(67ac975934dd179cd774fe6e58fb4c92) = 00003c8ac27a2e964c9c44a9f091fb53
24 花费时间: 2.98 秒
25 找到 21 个前导零, MD5(99feab89a09e27f689ea64b54d0ec154) = 000007cf815cba51f3c63e47ec30ccfb
26 花费时间: 6.77 秒
27 找到 22 个前导零, MD5(508c747156dc43b0b2fd238aab725f77) = 0000025860b5b803833c75ea1d53e9d9
28 花费时间: 21.07 秒

```

E 2.4-SHA1.txt	
1	找到 1 个前导零, SHA1(33) = 77de68daecd823babbb58edb1c8e14d7106e83bb
2	花费时间: 0.00 秒
3	找到 2 个前导零, SHA1(c4e74dddc9cc9e2fdcdb7f63b127fb638831262e) = 2f1cbe8d5f7f34529bbb60dd42868591567c253e
4	花费时间: 0.00 秒
5	找到 3 个前导零, SHA1(91da8b533f7ded74f27a8701dddf4cf816e95fa8) = 16255213344a663d5b5878d174f433ce86153563
6	花费时间: 0.00 秒
7	找到 6 个前导零, SHA1(5859803d38914a006a9a5736354edb37c8979c41) = 028635d0156b8287a173bd2b9e5a8acf5393ca28
8	花费时间: 0.00 秒
9	找到 8 个前导零, SHA1(a84251c0d77ed8cf16ab514226a83c941d6d3d79) = 00cd8636f87d23a8b39efdc41a05b6f9a5ec3fbf
10	花费时间: 0.00 秒
11	找到 9 个前导零, SHA1(f4d0b5fc05a31dd9d224b7b6ac18b57ff25c093c) = 007696db17b35ae8706d9b2b223807eb9a79f36c
12	花费时间: 0.00 秒
13	找到 10 个前导零, SHA1(8c453f0142701725d45f480080591a05a2fc9df6) = 003188f0f389daabeafa2a737dc849cca83588d8
14	花费时间: 0.01 秒
15	找到 11 个前导零, SHA1(dbef74dfbe42bf93d999f2c720fa01553356ba01) = 0019ee83628ef704ce3751c6688a78d489c574b1
16	花费时间: 0.01 秒
17	找到 14 个前导零, SHA1(367777f1b9ed7f7733b746409b14a674e0a586bc) = 000342e453dbdbb650e008e4f7db34c5a021e564
18	花费时间: 0.02 秒
19	找到 15 个前导零, SHA1(88cf20ed9f9277d4d939585b23bd7e0a9686a61b) = 0001563fbebe7743ca6c1576dcd1d320f91f1844
20	花费时间: 0.19 秒
21	找到 19 个前导零, SHA1(ecd0be2139b1770a4042669af973c78682a33872) = 0000122973896e5f43cd2f26e15f273af8862f22
22	花费时间: 0.28 秒
23	找到 21 个前导零, SHA1(86c88f32f205a9ef66bdf85a7220f59ca624d30b) = 000006cc121b427f95db4a9f041e2e83b6efe495
24	花费时间: 1.45 秒
25	找到 23 个前导零, SHA1(197c8c30dcc4497e1b0d8187a3022fd15081571e) = 0000011a17ce2230eefce65702cab5765bb28a78
26	花费时间: 4.49 秒

E 2.4-SHA256.txt	
1	找到 1 个前导零, SHA256(33) = 4e07408562bedb8b60ce05c1decfe3ad16b72230967de01f640b7e4729b49fce
2	花费时间: 0.00 秒
3	找到 2 个前导零, SHA256(c9ad7fd7952f47b722b34bc573736499b5035e5358a8f56d57b7aee51d7f6434) = 309418552956833b789524addf5793c36f4917cb6be6b8a1a3e7cad9255698f8
4	花费时间: 0.00 秒
5	找到 4 个前导零, SHA256(523b23952c44c8b8f74d193bee6124d639c017514c2960b965eefccc610527f1) = 0fd524c261ca3e0bda17f7e39e6d24f6708a30a630fefa83f8a8474233f58cf6
6	花费时间: 0.00 秒
7	找到 5 个前导零, SHA256(2578c04db7d8e9a8c73467d12b296d74a0cb9487eb55fe5514e1d9ffdf8f24e38) = 05639cef9bb0919af81cb1ecf2689aa415fe9cdc00203ac98ed08d93840b7068
8	花费时间: 0.00 秒
9	找到 6 个前导零, SHA256(fc5bf650dc2bd21e86f1750ec3292d6201f151fdb412e1c9ea5e962225644c86) = 03f04f5014ecf923f5ec1fab35bf536e8aeb6ce0833bdd220522e1c05cf35c1
10	花费时间: 0.00 秒
11	找到 8 个前导零, SHA256(eea50ac9ae77aaf82ad485617c389b24d8950ad10a8a891bc32585517688d0f7) = 00b4ca633ab01f00938af1982f6ad50727ae0217fae4b021f871d830a458709c
12	花费时间: 0.01 秒
13	找到 9 个前导零, SHA256(5439c7786c9394be63f6c238e88b95c4ce9a97f8933032f19921a44f58121f02) = 00792281a939c05b90a0d7cebe6338cc7a2d327a1e513446026f8158ca605a03
14	花费时间: 0.02 秒
15	找到 11 个前导零, SHA256(93aec93880b4fea67dc9d94593bc0d15dec3c42cb09c325ea51dad35e0fb632b) = 001eff029cb9ca4686bbd7666c985c404f78da5a582110bf6bf8060e7ca53581
16	花费时间: 0.02 秒
17	找到 14 个前导零, SHA256(d19fcc11888c54aaa374f4942ce15cedba6ac2906ab6f8d99de57665c22f5178c) = 000385955282d99dc77c9f9d83dd30c1f7ce51abd89803898ae16d7deae413eb
18	花费时间: 0.28 秒
19	找到 15 个前导零, SHA256(82a92de2e82a4beee3afec9445c2df0fbacd23a0282657cd72b7add05ac40331) = 00011321c3f2dbc7f089cf3de5d82161fb05b494d117bf1d2f224aa31f740dc
20	花费时间: 0.33 秒
21	找到 16 个前导零, SHA256(0000d8c33832a4e53d246778be2a393257e5262b084ebf837d54ba543b48655) = 00009edea8c2b22fb1eed7dc11d40895f46c9561f01108932e67dde1e96c17d
22	花费时间: 0.51 秒
23	找到 18 个前导零, SHA256(cd0d0e34de7b113abd8608311aeb17bb9265b8783ee8d2f846227147a3de8882) = 000039920f06c2abbb730432061d99d7d21e582f0139cdc964bc3dff45638f39
24	花费时间: 1.07 秒
25	找到 21 个前导零, SHA256(f0921a84acd6b0afc7d289ee6c80276d093e054f0f7b78935eddc7b55103c062) = 000006c7e1c99559f833b3a7982a1aeb6907985c64412f22c09b0e57d50ae9
26	花费时间: 9.45 秒
27	找到 22 个前导零, SHA256(9c0078b0e99f0d66c2f3e507757087ad5dc82aee663caf20a2f71696b44a421d) = 0000022f6b3c6be34161ecdef085dab5e6322cfc298ce710044777fd4b30875
28	花费时间: 38.34 秒

随着对前导零数量要求的增加, 找到满足条件的哈希值所需的尝试次数和时间呈指数级增长。这验证了哈希函数的单向特性, 即给定输出特征难以倒推输入, 只能通过大量搜索来寻找碰撞。